

Informe Final

Desafío 1 2026-2

Estudiante: Sergio Luis Laverde Florez

Cédula: 1062435422

Grupo: 2

Introducción: este es el último informe que se entrega en el contexto del primer desafío donde se plasma información sobre el análisis, desarrollo y evolución de la implementación del estudiante.

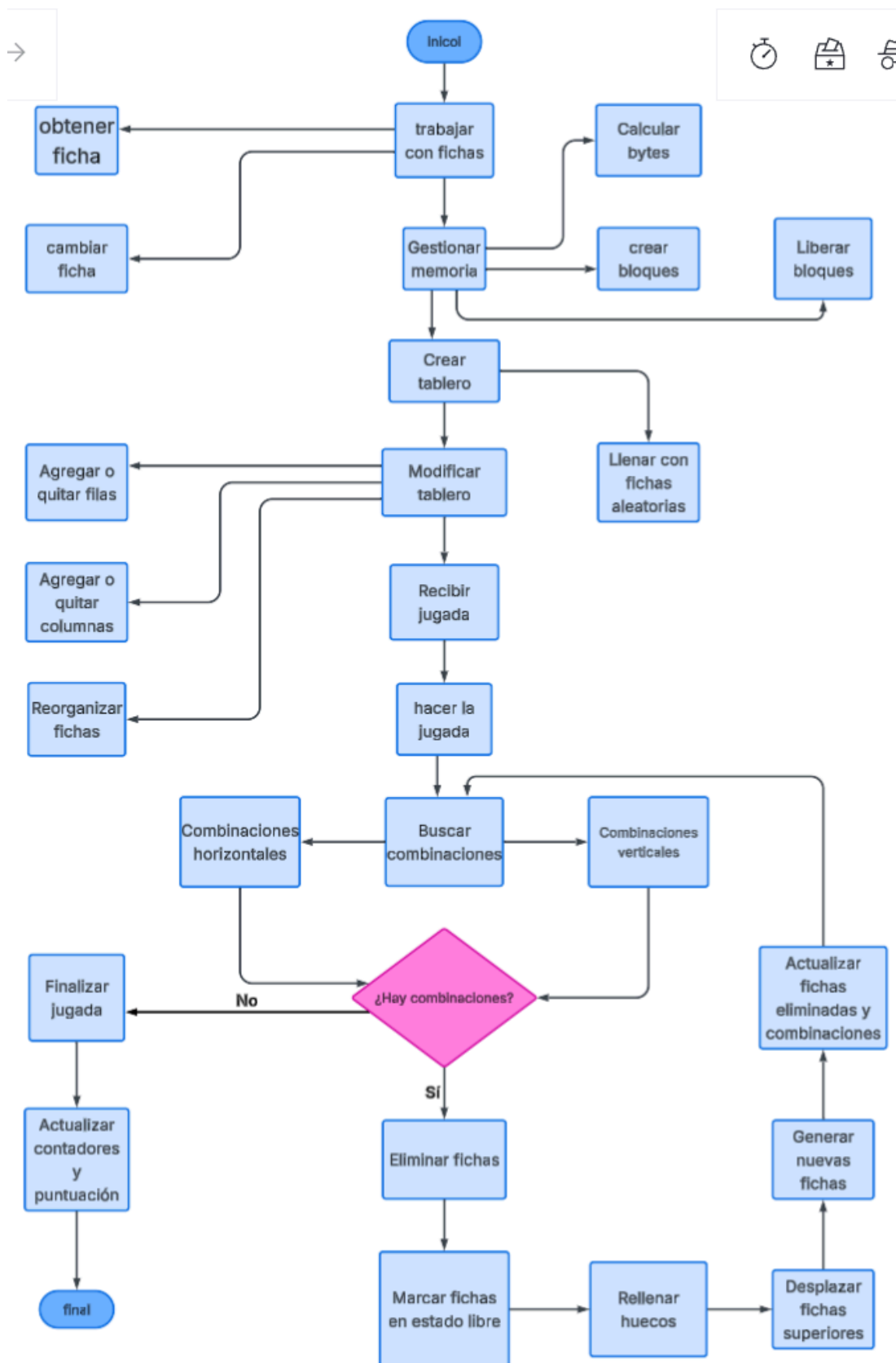
Análisis del problema y consideraciones para la alternativa de solución propuesta:

Desde un inicio se trabajó bajo la lógica o estrategia del “divide y vencerás” por que se distribuyo la carga analítica en varias partes dependiendo de cómo creía iba a avanzar mi implementación de la solución.

Se nos pedía desarrollar un juego del tipo sweet crush (candy crush) con las funcionalidades que por lo general traen este tipo de juegos (eliminación de combinaciones de fichas, caída en gravedad, generación aleatoria de ficha en la fila 0, combinaciones combinadas, cascadas. Lo de todos los juegos de este tipo), pero con ciertas diferencias:

1. no se hacen esos intercambios entre dos fichas, se elimina una en una posición y eso genera la caída que hace la dinámica del juego.
2. Se pueden hacer modificaciones en el tamaño del tablero, agregando o quitando filas y columnas(lo que nos puede llevar a redimensionamiento del sistema de memoria usado para gestionar el tablero).
3. Se debe implementar una interfaz de terminal en base a caracteres donde el usuario pueda jugar y realizar las modificaciones de las dimensiones del tablero.
4. En memoria cada ficha ocupa 3 bits y debe haber contigüidad entre estas, por lo que opte por un sistema de memoria dinámica de bloques de 6 bytes de memoria.

Esquema de las tareas definidas en el desarrollo de los algoritmos:



Algoritmos implementados

los dividiré en tres grupos: primarios (manejo de fichas individuales), tablero (redimensionamiento de tablero y memoria) y juego (combinaciones, e interfaz).

primarias:

obtener_ficha: algoritmo que busca en el sistema de memoria (en este caso de bloques dinámicos de 6 bytes) y aun con la contigüidad de los 3 bits logra apartar los tres bits deseados mediante máscaras y desplazamientos de bits.

cambiar_ficha: algoritmo que al entregarse una el valor numérico asociado a una ficha y una posición de otra ficha busca en el sistema de memoria dicha posición y sobrescribe los 3 bits del valor asociado en dicha posición.

tablero:

expandir_memoria: algoritmo que según los bytes necesarios para continuar con el juego y los bytes que ya tenemos reservados crea la cantidad de bloques de memoria justos para continuar el juego.

disminuir_memoria: algoritmo que según los bytes necesarios para continuar con el juego y los bytes que ya tenemos reservados destruye bloques de memoria para tener mejor eficiencia.

crear_tablero: algoritmo que crea un tablero nuevo usando fichas aleatorias y dimensiones ingresadas, generalmente aumenta la memoria usada por medio de expandir_memoria.

agregar_fila: algoritmo que si necesita memoria llama a aumentar_memoria y por medio de los algoritmos primarios reorganiza el tablero sobre poniendo fichas una sobre otra de tal manera que una fila quede libre en el lugar deseado y se mantenga el orden del tablero solo que con la nueva fila llena de fichas en estado libre

agregar_columna: algoritmo que por medio de los algoritmos primarios reorganiza el tablero creando un tablero nuevo usando el mismo sistema de memoria y sobreponiendo las fichas ya ordenadas del tablero anterior sobre este nuevo dejando la fila deseada con ficha en estado libre (también desplazo las columnas posicionadas después de la que insertamos incluyendo a la misma).

quitar_fila: algoritmo que por medio de los algoritmos primarios reorganiza el tablero sobre poniendo fichas una sobre otra de tal manera que la fila deseada quede eliminada y se mantenga el orden del tablero solo que sin la fila, al final mira si se usa el 65% de la memoria reservada y en base a esto llama o no a disminuir memoria.

quitar columna: algoritmo que por medio de los algoritmos primarios reorganiza el tablero creando un tablero nuevo usando el mismo sistema de memoria y sobreponiendo las fichas ya ordenadas del tablero anterior sobre este de tal manera que el nuevo tablero quede con el mismo orden del tablero anterior pero sin la columna a quitar, antes de esto verifica si su usa el 65% de la memoria reservada para saber el tamaño en memoria del nuevo tablero.

juego:

print_tablero: algoritmo que por medio de los algoritmos primarios recorre las fichas del tablero según la ficha imprime un caracter en la terminal, todo esto de forma ordenada para la interfaz del juego

(aquí agrupare 4 algoritmos porque son prácticamente lo mismo):

buscar _ arriba/abajo/izquierda/derecha: algoritmos que por medio de los algoritmos primarios buscan en una dirección las fichas que son iguales y dan el número de fichas iguales que encontraron respecto a una ficha.

buscar_combinaciones: algoritmo que por medio de los algoritmos primarios y los de buscar ya mostrados recorre las fichas del tablero y en cada una verifica el número de fichas iguales en todas las direcciones y si encuentra cambia el valor de las fichas pertenecientes a las combinaciones a estado libre.

rellenar_huecos: algoritmo que por medio de los algoritmos primarios recorre las fichas del tablero en busca de fichas en estado libre, una vez las encuentra busca las fichas arriba de esta de una en una hasta encontrar una ficha con una valor diferente a estado libre o el techo superior y toma el valor de esa ficha y se la da, si llega al techo superior toma un valor random , recorre el tablero desde filas inferiores hasta las superiores para garantizar que todas las fichas queden con valor diferente a estado libre.

quitar_una_ficha: algoritmo que por medio los índices de una ficha la vuelve estado libre y baja cada una de las fichas arriba de ella una fila y a la más alta le brinda un valor random (no usamos rellenar huecos por eficiencia).

mostrar_juego: algoritmo final y el de más alto nivel, por medio de uso de todos los algoritmos ya mostrados anteriormente imprime el tablero inicial al invocar este algoritmo y de manera cíclica buscar el número de combinaciones, imprime el tablero, las rellena, vuelve a imprimir el tablero, vuelve buscar combinaciones... y así hasta que el tablero entre en un equilibrio y no encuentre más combinaciones, al final le muestra al usuario datos del juego como número de fichas eliminadas, combinaciones puntos.. etc.

main: no es como tal un algoritmo si no el todo del programa pero es el que usa todos los algoritmos anteriores más las entradas del usuario en la interfaz para hacer que el juego se comporte de manera correcta respecto a la jugabilidad.

problemas afrontados en el desarrollo:

1. accesos inválidos en la memoria: debido a que use un sistema de memoria dinámica de bloques de 6 bytes y además las fichas pueden estar repartidas entre dos bytes se me generaron muchos problemas por acceder a memoria cuando no me era necesario, en especial en los casos donde los bloques se llenaban completamente (al crear tableros $n*n$ donde n es un múltiplo de 6) y se solicitaba acceso a un byte que no existía. se soluciono por medio de condicionales que verificaban la necesidad de acceder al respectivo byte.
2. problemas de índices: al hacer las funciones de agregar y quitar columna(que fueron de las últimas) me di cuenta que no se podían trabajar con los mismos índices que dictan la posición del primer bit del elemento pues se modifica el número de columnas el cual era necesario para la saber el indice por lo que dentro de mi sistema de memoria no se podía modificar la tabla ya hecha como si se podia en agregar y quitar filas. se soluciono planteando el problema desde cero y creando un tablero aparte y ahora si a partir del primero se organizaba el segundo.
3. mi compañero no aportó nada al proyecto y yo tomé toda la carga. Sin embargo se logró terminar el proyecto con esfuerzo y dedicación.

Evolución de la solución:

en un primer momento se planteó una solución para la parte del nivel de bits y como obtener la información de estos, al mismo tiempo se plantaron los algoritmos del juego en sí de hallar combinaciones rellenar huecos además del uso de las fichas restantes donde escogí no usar una de estas y usar la otra como estado libre, después a partir de las funciones de operaciones a nivel de bits que ya mencione cree la lógica para las alteraciones del tablero y sus implicaciones en la memoria de esto ultimo decidi apartar las funciones de gestión de memoria.

En si fue un proceso constante de hacer un análisis a primera con la información que tenga previamente y en base a el empezar a hacer código para que en el proceso las ideas fluyan, ese es en gran parte mi modo de programar.